

Bridge GeCode - Manual Completo

Guía Completa de Sets y Operaciones

Proyecto GNUBison

2026

Contents

Introducción	3
¿Qué es Bridge GeCode?	3
Características Principales	3
Tipos de Datos Soportados	3
Operadores	3
Arquitectura del Sistema	5
Componentes Principales	5
Archivos del Proyecto	5
Compilación e Instalación	6
Requisitos	6
Instalación de Dependencias	6
Compilación	6
Verificación	6
Estructura de Directorios	6
Ejemplo 1: Expresión Aritmética Simple	7
Caso de Uso	7
JSON de Entrada	7
Comando	7
JSON de Salida	7
Análisis del Resultado	8
Ejemplo 2: Incertidumbre con Múltiples Valores	9
Caso de Uso	9
JSON de Entrada	9
Comando	9
JSON de Salida	9
Análisis Detallado	10
Ejemplo 3: Sistema de Control con Sensores	12
Caso de Uso	12
JSON de Entrada	12

Comando	12
JSON de Salida	12
Interpretación de Resultados	13
Ejemplo 4: Constraint Satisfaction Problem (CSP)	14
Caso de Uso	14
JSON de Entrada	14
Análisis de Restricciones	14
Validación del Sistema	14
Resumen de Capacidades	15
Tipos de Evaluación	15
Precisión Numérica	15
Manejo de Incertidumbre	15
Casos de Uso	15
Operaciones de Conjuntos	16
Introducción	16
Operador IN	16
UNION	17
INTERSECT	18
DIFFERENCE	19
SUBSET	20
CARDINALITY	21
Ejemplos Completos	22
Casos de Uso Comunes	24
Combinando con Otros Operadores	25
Notas Importantes	25
Resumen de Operadores	25

Introducción

¿Qué es Bridge GeCode?

Bridge GeCode es un compilador y evaluador de expresiones diseñado para problemas de **Constraint Programming (CSP)**. Funciona como un pipeline que:

1. **Recibe** especificaciones en formato JSON o Pascal-like
2. **Valida** la sintaxis y semántica
3. **Evalúa** expresiones con variables
4. **Retorna** resultados en formato JSON

Características Principales

- **Validación de Sintaxis:** Parser robusto con Flex/Bison
- **Evaluación de Expresiones:** Calcula resultados automáticamente
- **Soporte Dual:** Acepta formato JSON y Pascal-like
- **Manejo de Incertidumbre:** Variables con múltiples valores posibles
- **Operaciones de Conjuntos:** UNION, INTERSECT, DIFFERENCE, SUBSET, CARDINALITY, IN
- **Sintaxis Moderna de Sets:** {a,b,c} sin comillas en expresiones
- **Salida Estructurada:** JSON con resultados detallados
- **Funciones Matemáticas:** abs, sqrt, sin, cos, ln, exp, etc.

Tipos de Datos Soportados

Tipo	Descripción	Ejemplo
integer	Números enteros	[1, 100]
float	Números decimales	[0.0, 50.0]
logic	Booleanos	[true, false]
set	Conjuntos de strings	["A", "B", "C"]

Operadores

Aritméticos

+, -, *, /

Lógicos

AND, OR, NOT, IMPLICA

Relacionales

=, <>, <, >, <=, >=

Operadores de Conjuntos

- **IN:** Verificar pertenencia $\rightarrow x \text{ IN } \{a,b,c\}$
- **UNION:** Unión de conjuntos $\rightarrow A \text{ UNION } B$

- **INTERSECT**: Intersección $\rightarrow A \text{ INTERSECT } B$
- **DIFFERENCE**: Diferencia $\rightarrow A \text{ DIFFERENCE } B$
- **SUBSET**: Verificar subconjunto $\rightarrow A \text{ SUBSET } B$
- **CARDINALITY**: Contar elementos $\rightarrow \text{CARDINALITY}(A)$

Funciones Estándar

`abs(x)`, `sqrt(x)`, `sqr(x)`, `sin(x)`, `cos(x)`, `ln(x)`, `exp(x)`

Arquitectura del Sistema

Componentes Principales



Archivos del Proyecto

- **cJSON.c**: Biblioteca JSON (MIT License)

Compilación e Instalación

Requisitos

- **gcc** (compilador C)
- **bison** ≥ 3.0
- **flex** ≥ 2.6
- **make**

Instalación de Dependencias

Ubuntu/Debian

```
sudo apt install gcc bison flex make
```

Fedora/RHEL

```
sudo dnf install gcc bison flex make
```

Compilación

Clonar o descargar el proyecto

```
cd GNUBison
```

Compilar

```
make
```

Limpiar archivos generados

```
make clean
```

Verificación

Ejecutar ejemplo simple

Debe mostrar JSON con resultados

Estructura de Directorios

GNUBison/

Ejemplo 1: Expresión Aritmética Simple

Caso de Uso

Evaluar una expresión aritmética básica: $(x + y) * z$

JSON de Entrada

```
{
  "precision": 2,
  "variables": [
    {"nombre": "x", "tipo": "integer", "domain": [1, 100], "value": 10},
    {"nombre": "y", "tipo": "integer", "domain": [1, 100], "value": 20},
    {"nombre": "z", "tipo": "float", "domain": [0.0, 50.0], "value": 15.5}
  ],
  "expresiones": [
    "(x + y) * z"
  ]
}
```

Comando

```
./bridge ejemplo_simple.json --json-output
```

JSON de Salida

```
{
  "archivo_entrada": "ejemplo_simple.json",
  "precision": 2,
  "factor": 100,
  "variables": [
    {
      "nombre": "x",
      "tipo": "integer",
      "dominio": [1, 100],
      "valor": 10
    },
    {
      "nombre": "y",
      "tipo": "integer",
      "dominio": [1, 100],
      "valor": 20
    },
    {
      "nombre": "z",
      "tipo": "float",
      "dominio": [0, 50],
      "valor": 15.5
    }
  ]
}
```

```
],
"expresiones": [
  {
    "expresion": "(x + y) * z",
    "resultado": 46500
  }
],
"resumen": {
  "total_variables": 3,
  "total_expresiones": 1,
  "errores": 0,
  "valido": true
}
}
```

Análisis del Resultado

Cálculo Manual: - $x = 10$ - $y = 20$ - $z = 15.5$ - $(x + y) * z = (10 + 20) * 15.5 = 30 * 15.5 = 465$

Resultado del Sistema: - Valor retornado: 46500 - Precisión: 2 decimales (factor = 100) -
Conversión: $46500 / 100 = 465.00$

Ejemplo 2: Incertidumbre con Múltiples Valores

Caso de Uso

Evaluar expresiones cuando las variables tienen **múltiples valores posibles** (incertidumbre).

JSON de Entrada

```
{
  "precision": 1,
  "variables": [
    {"nombre": "a", "tipo": "integer", "domain": [1, 10], "value": [2, 5, 8]},
    {"nombre": "b", "tipo": "integer", "domain": [1, 10], "value": 3},
    {"nombre": "valido", "tipo": "logic", "domain": [true, false],
      "value": [true, false]}
  ],
  "expresiones": [
    "a + b",
    "a > b",
    "valido AND a > 3"
  ]
}
```

Comando

```
./bridge ejemplo_incertidumbre.json --json-output
```

JSON de Salida

```
{
  "archivo_entrada": "ejemplo_incertidumbre.json",
  "precision": 1,
  "factor": 10,
  "variables": [
    {
      "nombre": "a",
      "tipo": "integer",
      "dominio": [1, 10],
      "valor": [2, 5, 8]
    },
    {
      "nombre": "b",
      "tipo": "integer",
      "dominio": [1, 10],
      "valor": 3
    },
    {
      "nombre": "valido",
      "tipo": "logic",

```

```

        "dominio": [true, false],
        "valor": [true, false]
    }
],
"expresiones": [
    {
        "expresion": "a + b",
        "resultado": [5, 8, 11]
    },
    {
        "expresion": "a > b",
        "resultado": [false, true, true]
    },
    {
        "expresion": "valido AND a > 3",
        "resultado": [true, true, true, false, false, false]
    }
],
"resumen": {
    "total_variables": 3,
    "total_expresiones": 3,
    "errores": 0,
    "valido": true
}
}

```

Análisis Detallado

Expresión 1: $a + b$

a	b	Resultado
2	3	5
5	3	8
8	3	11

Salida: [5, 8, 11]

Expresión 2: $a > b$

a	b	$a > b$
2	3	false
5	3	true
8	3	true

Salida: [false, true, true]

Expresión 3: `valido AND a > 3`

Producto cartesiano de todas las combinaciones:

valido	a	a > 3	valido AND (a>3)
true	2	false	false → true
true	5	true	true
true	8	true	true
false	2	false	false
false	5	true	false
false	8	true	false

Salida: `[true, true, true, false, false, false]`

Ejemplo 3: Sistema de Control con Sensores

Caso de Uso

Sistema de monitoreo con temperatura, presión y estado activo.

JSON de Entrada

```
{
  "precision": 1,
  "variables": [
    {"nombre": "temperatura", "tipo": "float",
     "domain": [0.0, 100.0], "value": 25.5},
    {"nombre": "presion", "tipo": "integer",
     "domain": [1, 10], "value": [3, 5, 7]},
    {"nombre": "activo", "tipo": "logic",
     "domain": [true, false], "value": true}
  ],
  "expresiones": [
    "temperatura > 20.0",
    "presion * 2",
    "activo AND temperatura > 15.0"
  ]
}
```

Comando

```
./bridge pipeline_demo.json --json-output
```

JSON de Salida

```
{
  "archivo_entrada": "pipeline_demo.json",
  "precision": 1,
  "factor": 10,
  "variables": [
    {
      "nombre": "temperatura",
      "tipo": "float",
      "dominio": [0, 100],
      "valor": 25.5
    },
    {
      "nombre": "presion",
      "tipo": "integer",
      "dominio": [1, 10],
      "valor": [3, 5, 7]
    },
    {

```

```

        "nombre": "activo",
        "tipo": "logic",
        "dominio": [true, false],
        "valor": true
    }
],
"expresiones": [
    {
        "expresion": "temperatura > 20.0",
        "resultado": true
    },
    {
        "expresion": "presion * 2",
        "resultado": [6, 10, 14]
    },
    {
        "expresion": "activo AND temperatura > 15.0",
        "resultado": true
    }
],
"resumen": {
    "total_variables": 3,
    "total_expresiones": 3,
    "errores": 0,
    "valido": true
}
}

```

Interpretación de Resultados

Temperatura dentro de rango

- Expresión: temperatura > 20.0
- Valor: 25.5 °C
- **Resultado:** true → Sistema operando correctamente

Presión con incertidumbre

- Expresión: presion * 2
- Valores posibles: [3, 5, 7]
- **Resultados:** [6, 10, 14] → Múltiples escenarios

Sistema activo y temperatura adecuada

- Expresión: activo AND temperatura > 15.0
- **Resultado:** true → Condiciones óptimas

Ejemplo 4: Constraint Satisfaction Problem (CSP)

Caso de Uso

Asignación de recursos con restricciones.

JSON de Entrada

```
{
  "precision": 1,
  "variables": [
    {"nombre": "enfermeros", "tipo": "integer",
     "domain": [2, 15], "value": null},
    {"nombre": "medicos", "tipo": "integer",
     "domain": [1, 8], "value": [3, 4, 5]},
    {"nombre": "urgente", "tipo": "logic",
     "domain": [true, false], "value": true}
  ],
  "expresiones": [
    "enfermeros + medicos <= 20",
    "enfermeros >= medicos * 2",
    "urgente = 1 IMPLICA medicos >= 3"
  ]
}
```

Análisis de Restricciones

Restricción 1: Total de personal limitado

$\text{enfermeros} + \text{medicos} \leq 20$

Restricción 2: Proporción enfermeros/médicos

$\text{enfermeros} \geq \text{medicos} * 2$

Al menos 2 enfermeros por cada médico

Restricción 3: Mínimo de médicos en emergencias

$\text{urgente} = 1 \text{ IMPLICA } \text{medicos} \geq 3$

Si es urgente, mínimo 3 médicos

Validación del Sistema

El evaluador calcula **todas las combinaciones posibles** y verifica qué asignaciones satisfacen las restricciones.

Ventajas: - Verifica consistencia de restricciones - Identifica valores que satisfacen todas las condiciones - Detecta contradicciones automáticamente

Resumen de Capacidades

Tipos de Evaluación

Característica	Soportado	Ejemplo
Valores fijos		"value": 10
Incertidumbre		"value": [10, 20, 30]
Sin asignación		"value": null
Expresiones anidadas		(a + b) * (c - d)
Funciones matemáticas		abs(x - y)
Lógica booleana		A AND (B OR C)

Precisión Numérica

El sistema maneja precisión decimal configurable:

- **precision: 0** → Factor = 1 (enteros)
- **precision: 1** → Factor = 10 (1 decimal)
- **precision: 2** → Factor = 100 (2 decimales)
- **precision: 3** → Factor = 1000 (3 decimales)

Los valores se almacenan como enteros multiplicados por el factor.

Ejemplo: - Entrada: 15.5 con precision=1 - Almacenamiento: 155 (factor=10) - Salida: 15.5

Manejo de Incertidumbre

Cuando hay variables con múltiples valores, el sistema:

1. Calcula el **producto cartesiano** de todas las combinaciones
2. Evalúa la expresión para **cada combinación**
3. Retorna **array con todos los resultados**

Complejidad: - n variables con m valores cada una - Total combinaciones: m^n - Todas las combinaciones se evalúan

Casos de Uso

- **Validación de restricciones** en CSP
- **Análisis de sensibilidad** con incertidumbre
- **Verificación formal** de condiciones lógicas
- **Cálculo simbólico** con evaluación numérica
- **Pruebas automatizadas** de expresiones
- **Sistemas expertos** con reglas lógicas

Operaciones de Conjuntos

Introducción

Bridge GeCode soporta operaciones avanzadas de conjuntos (sets) que permiten modelar problemas complejos de asignación, permisos, roles, y más.

Sintaxis Moderna

Importante: Los sets literales en expresiones se escriben **sin comillas**:

Correcto: {dev,qa,pm}

Incorrecto: {"dev","qa","pm"}

Nota: En JSON, los valores de variables SÍ llevan comillas por estándar JSON:

```
"value": ["dev", "qa"]
```

Operador IN

Verifica si un elemento pertenece a un conjunto.

Sintaxis

elemento IN {conjunto}

variable IN otra_variable

Ejemplos

Verificar pertenencia de número:

codigo IN {10,20,30,40,50}

Verificar pertenencia de identificador:

rol IN {admin,supervisor,gerente}

Con variable:

usuario IN equipo_autorizado

UNION

Combina dos conjuntos, retornando todos los elementos únicos.

Sintaxis

```
conjuntoA UNION conjuntoB  
variable UNION {elementos}
```

Ejemplos

Agregar roles:

```
equipo UNION {pm,dba}
```

Si equipo = {dev,qa}, resultado: {dev,qa,pm,dba}

Combinar permisos:

```
permisos_base UNION permisos_extra
```

INTERSECT

Retorna solo los elementos comunes entre dos conjuntos.

Sintaxis

```
conjuntoA INTERSECT conjuntoB
```

Ejemplos

Encontrar roles comunes:

```
equipo INTERSECT {dev,qa,admin}
```

Si equipo = {dev,qa,pm}, resultado: {dev,qa}

Servicios activos y permitidos:

```
servicios_activos INTERSECT servicios_permitidos
```

DIFFERENCE

Retorna elementos del primer conjunto que NO están en el segundo.

Sintaxis

`conjuntoA DIFFERENCE conjuntoB`

Ejemplos

Remover elementos:

`equipo DIFFERENCE {qa}`

Si `equipo = {dev,qa,pm}`, resultado: `{dev,pm}`

Tareas pendientes:

`tareas_totales DIFFERENCE tareas_completadas`

SUBSET

Verifica si el primer conjunto es subconjunto del segundo (todos sus elementos están contenidos).

Sintaxis

```
conjuntoA SUBSET conjuntoB
```

Ejemplos

Verificar permisos mínimos:

```
{read,write} SUBSET permisos
```

Retorna true si permisos contiene al menos read y write.

Verificar roles requeridos:

```
roles_minimos SUBSET roles_asignados
```

Comparación:

```
{dev} SUBSET equipo          → true si equipo contiene dev  
{dev,admin} SUBSET equipo    → true solo si equipo contiene AMBOS
```

CARDINALITY

Cuenta la cantidad de elementos en un conjunto.

Sintaxis

```
CARDINALITY(conjunto)
CARDINALITY({elementos})
```

Ejemplos

Contar elementos de variable:

```
CARDINALITY(equipo)
```

Si equipo = {dev,qa,pm}, resultado: 3

Contar literal:

```
CARDINALITY({alice,bob,charlie})
```

Resultado: 3

En restricciones:

```
CARDINALITY(equipo) >= 3
```

```
CARDINALITY(permisos UNION {admin}) <= 5
```

Ejemplos Completos

Ejemplo 1: Control de Acceso

Archivo: ejemplo_acceso.json

```
{
  "precision": 0,
  "variables": [
    {
      "nombre": "permisos",
      "tipo": "set",
      "domain": ["read","write","execute","delete"],
      "value": ["read", "write"]
    },
    {
      "nombre": "autenticado",
      "tipo": "logic",
      "domain": [true, false],
      "value": true
    }
  ],
  "expresiones": [
    "autenticado = true",
    "{read} SUBSET permisos",
    "{write,delete} SUBSET permisos",
    "CARDINALITY(permisos) >= 2"
  ]
}
```

Ejecutar:

```
./bridge ejemplo_acceso.json resultado.json
```

Resultado:

```
{
  "expresiones": [
    {"expression": "autenticado = true", "resultado": true},
    {"expression": "{read} SUBSET permisos", "resultado": true},
    {"expression": "{write,delete} SUBSET permisos", "resultado": false},
    {"expression": "CARDINALITY(permisos) >= 2", "resultado": true}
  ]
}
```

Ejemplo 2: Gestión de Proyectos

Archivo: ejemplo_proyecto.json

```
{
  "precision": 0,
  "variables": [
    {
      "nombre": "equipoActual",
      "tipo": "set",
      "domain": ["dev", "qa", "pm", "ops"],
      "value": ["dev", "qa"]
    },
    {
      "nombre": "equipoNecesario",
      "tipo": "set",
      "domain": ["dev", "qa", "pm", "ops"],
      "value": ["qa", "pm", "ops"]
    }
  ],
  "expresiones": [
    "CARDINALITY(equipoActual) >= 2",
    "CARDINALITY(equipoActual UNION equipoNecesario) <= 4",
    "equipoActual INTERSECT equipoNecesario"
  ]
}
```

Resultado:

```
{
  "expresiones": [
    {"expresion": "CARDINALITY(equipoActual) >= 2", "resultado": true},
    {"expresion": "CARDINALITY(equipoActual UNION equipoNecesario) <= 4", "resultado": true},
    {"expresion": "equipoActual INTERSECT equipoNecesario", "resultado": ["qa"]}
  ]
}
```

Casos de Uso Comunes

1. Control de Acceso y Permisos

```
{admin} SUBSET roles  
permisos UNION {read,write}  
permisos DIFFERENCE {delete}
```

2. Gestión de Equipos

```
CARDINALITY(equipo) >= 3  
equipo INTERSECT roles_senior  
{dev,qa} SUBSET equipo
```

3. Configuración de Servicios

```
servicio IN {http,https,ssh}  
servicios_activos UNION {ftp}  
servicios DIFFERENCE servicios_deprecated
```

4. Recursos Humanos

```
departamento IN {ventas,it,rrhh}  
CARDINALITY(idiomas) >= 2  
certificaciones SUBSET {iso9001,iso27001}
```

5. Inventarios y Productos

```
producto IN stock_disponible  
categorias UNION {electronicos}  
CARDINALITY(productos) <= 100
```

Combinando con Otros Operadores

Con Operadores Lógicos

```
activo = true AND {admin} SUBSET roles  
CARDINALITY(equipo) >= 3 OR urgente = true
```

Con Operadores Aritméticos

```
CARDINALITY(equipo) * costo_persona <= presupuesto  
empleados >= CARDINALITY(departamentos) * 5
```

Con Comparaciones

```
CARDINALITY(servicios UNION {nuevo}) <= 10  
CARDINALITY(permisos) <> 0
```

Notas Importantes

1. **Sets vacíos:** {} es un conjunto válido con cardinalidad 0
2. **Elementos únicos:** Los sets no permiten duplicados
3. **Tipos:** Los elementos de un set son siempre strings o identificadores
4. **Comparación:** Dos sets son iguales si contienen exactamente los mismos elementos
5. **Performance:** Las operaciones de sets son eficientes incluso con conjuntos grandes

Resumen de Operadores

Operador	Símbolo	Tipo Retorno	Ejemplo
IN	IN	logic	x IN {a,b,c}
UNION	UNION	set	A UNION B
INTERSECT	INTERSECT	set	A INTERSECT B
DIFFERENCE	DIFFERENCE	set	A DIFFERENCE B
SUBSET	SUBSET	logic	A SUBSET B
CARDINALITY	CARDINALITY	integer	CARDINALITY(A)